

程序设计实践

The Practice of Programming

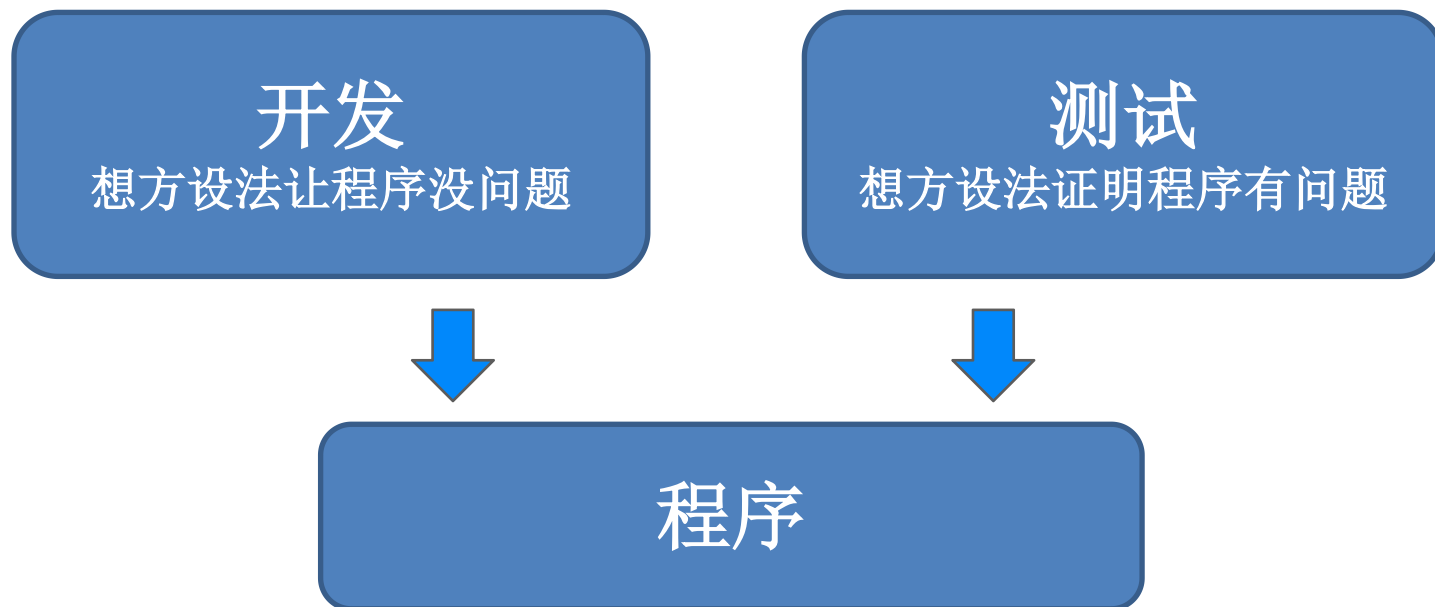
测试(Testing)

什么是测试

因为，“**BUG**越晚解决，解决它所需成本越高”

所以，测试的目的，则是尽早发现**BUG**，以便尽早解决它。

测试只能证明程序存在**BUG**，并不能证明程序不存在**BUG**。

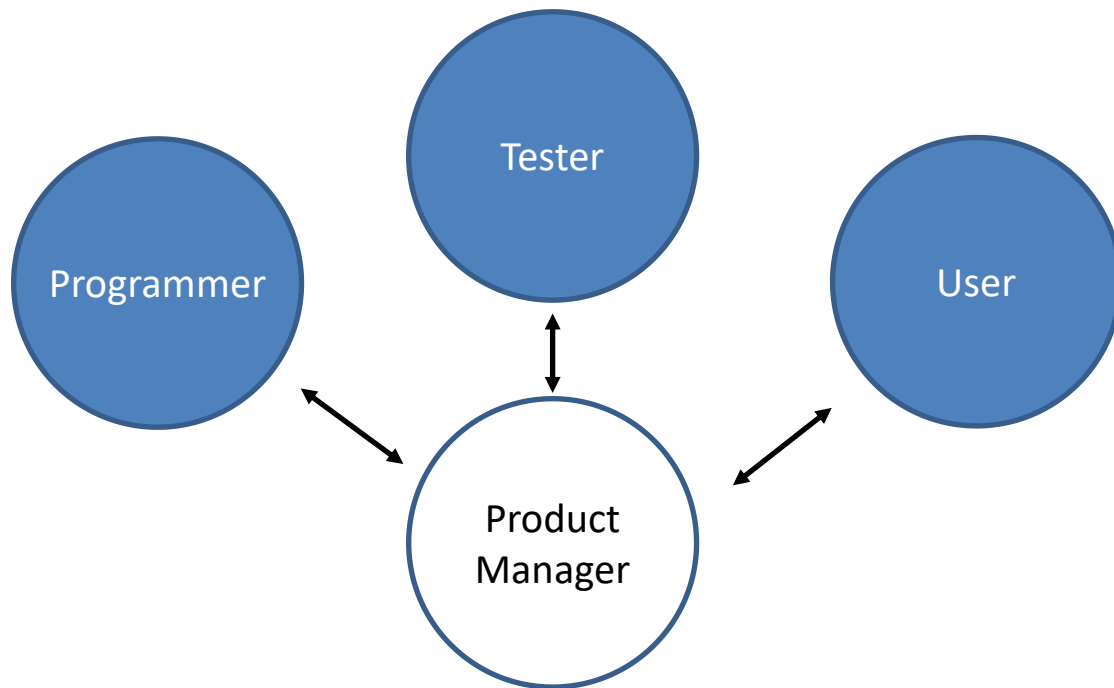


谁来测试

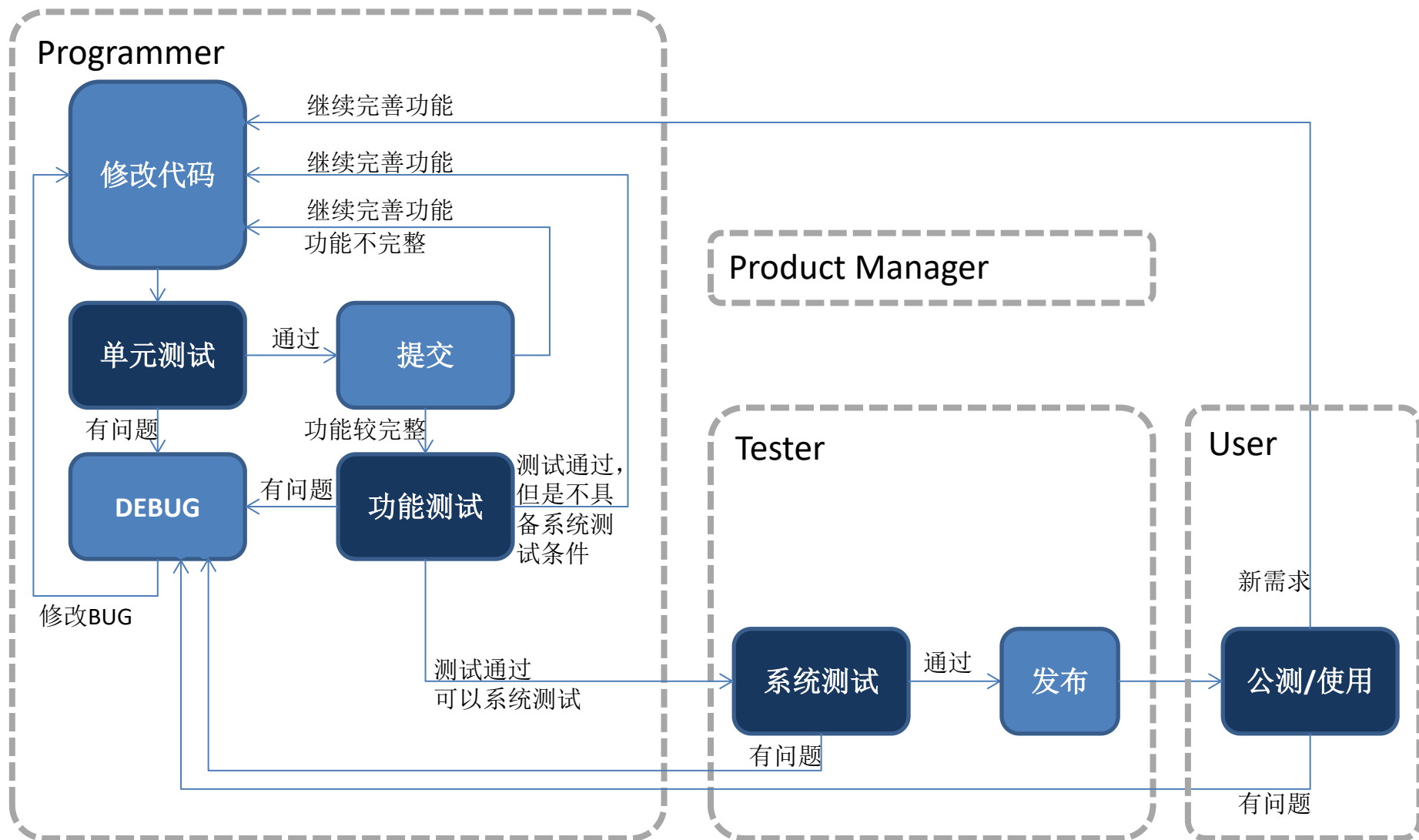
开发人员，应该首先做完善的测试，通过各种手段，发现程序的绝大部分的**BUG**，并解决它们。

相对完整的开发机构，会有单独的测试部门，来负责系统测试。

有的软件产品，会允许一定范围内的试用用户进行内测，公测。



测试的各个环环节



本章要点

1. 编写测试驱动，测试桩
2. 测试集的构建
3. 测试自动化(**Test Automation**)
4. 性能测试(**Performance Testing**)
5. 压力测试(**Stress Testing**)

测试驱动和测试桩



开发人员应该及时为自己负责的模块准备测试驱动和测试桩，自己搭建测试环境。

决不能依赖合作伙伴负责的模块开发好再开始测试。

```
/* reverse string, "abcde" -> "edcba" */  
void str_reverse(char* s)  
{  
    ...  
}
```

```
char s[256];  
strcpy(s, "abcde");  
str_reverse(s);  
printf("%s\n", s);
```

```
char s[256];  
strcpy(s, "abcde");  
str_reverse(s);  
ASSERT(strcmp(s, "edcba") == 0);
```

```
#define ASSERT(x) \  
    if (!(x)) { \  
        printf("%s:%d: ASSERT FAIL\n", __FILE__, __LINE__); \  
        exit(1) \  
    }
```

```
#ifdef _UNIT_TEST_
void test_str_reverse(s1, s2)
{
    char s[256];
    ASSERT(strlen(s1) == strlen(s2) && strlen(s1) < sizeof(s));
    strcpy(s, s1);
    str_reverse(s);
    ASSERT(strcmp(s, s2) == 0);
}
void main() /* 用于单元测试的main函数 */
{
    test_str_reverse("abcde", "edcba");
    test_str_reverse("abcd", "dcba");
    test_str_reverse("a", "a");
    test_str_reverse("", "");
}
#endif
```

```
#ifndef _UNIT_TEST_
void main() /* 真正的main函数 */
{
    ...
}
#endif
```


如果你的程序的逻辑是这样：

...
获得当前月份
根据当前月份构造url
发起http请求
获取当前月份的节假日表
判断http请求的返回结果是否正确
根据返回结果，继续运行后续逻辑
...

你的程序需要一个**http server**提供一个服务，根据月份查询节假日，接口大致这样：

请求：

`http://x.x.x.x:xxxx/Holidays?m=202110`

返回：

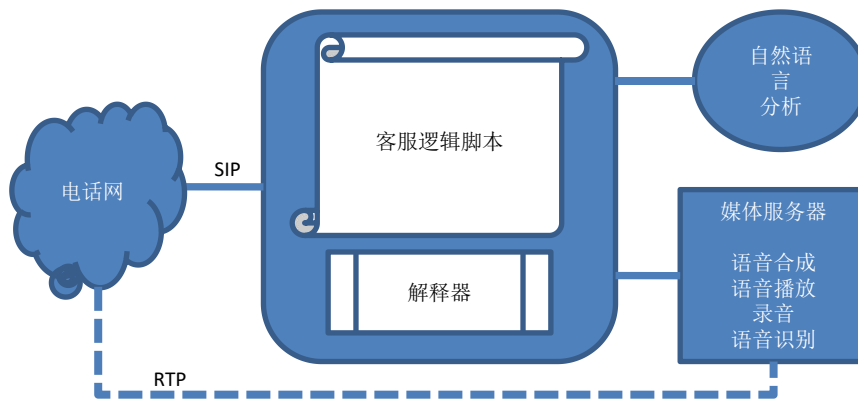
```
{
  "code":0,
  "info":{
    "202110":{
      "01":"1", "02":"1", "03":"1",
      "04":"1", "05":"1", "06":"1",
      "07":"1", "10":"1", "16":"1",
      "17":"1", "23":"1", "24":"1",
      "30":"1", "31":"1"
    }
  }
}
```

这个**http server**是合作伙伴负责实现的。

这时就需要自己搭建一个**http server**作为测试桩。

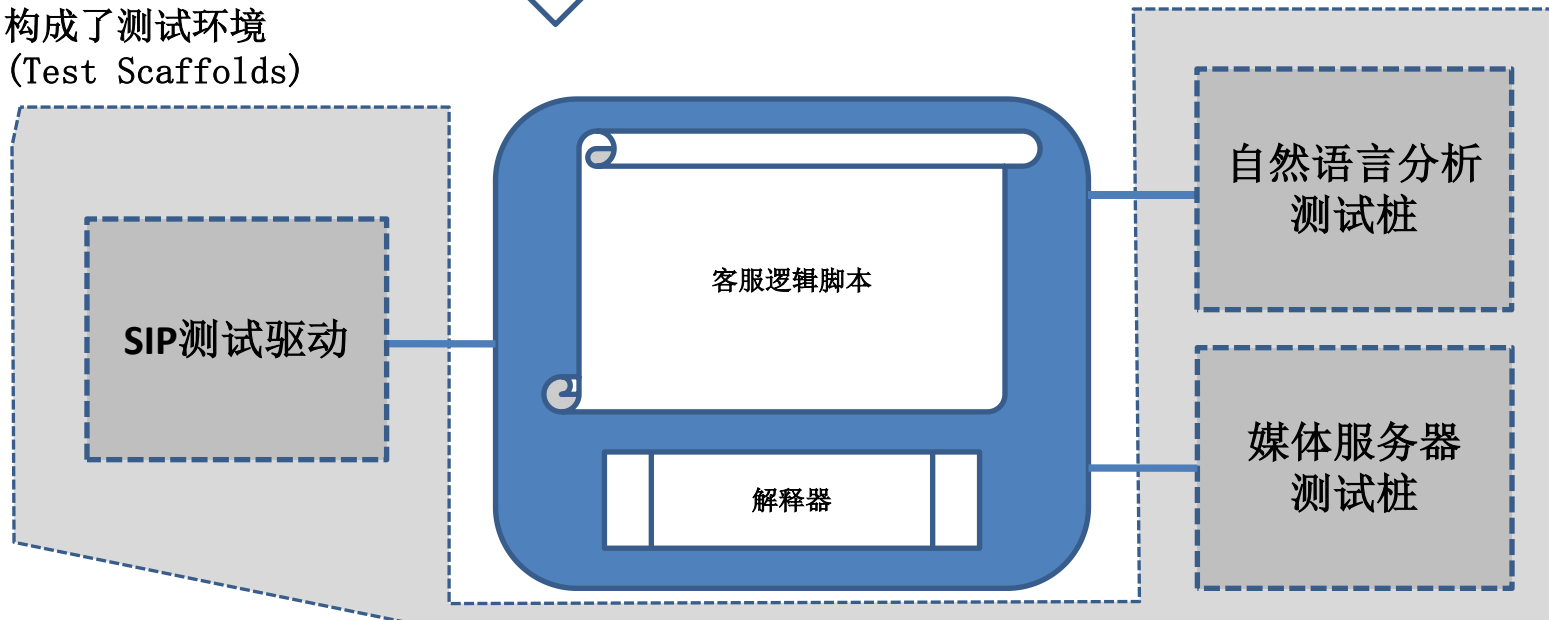
测试桩可以根据需要返回正确的，或者错误的结果，以便测试正常和异常分支。

所以，测试桩的作用不仅仅是替代合作伙伴还没做好的**http server**，还有一个重要的作用是，它能够根据你的需要返回任何结果。



一个较为复杂的系统，必须自己构建各种测试桩模拟周边的实体，让你的程序能够在发布之前得到有效的测试。

测试驱动和测试桩一起
构成了测试环境
(Test Scaffolds)



脚手架(Scaffolds)



命令行驱动: `robot test1.rsl name=张先生 bill=100`

```
Step welcome
Speak $name + "您好, 请问有什么可以帮您?"
Listen 5 20
Branch "故障", complainProc
Branch "账单", billProc
Silence silence
Default defaultProc
Step complainProc
Speak "您的意见是我们改进工作的动力, 请问您还有什么补充?"
Listen 5 50
Default thanks
Step thanks
Speak "感谢您的来电, 再见"
Exit
Step billProc
Speak "您当前的月账单是: " + $amount + "元, 感谢您的来电, 再见"
Exit
Step defaultProc
```

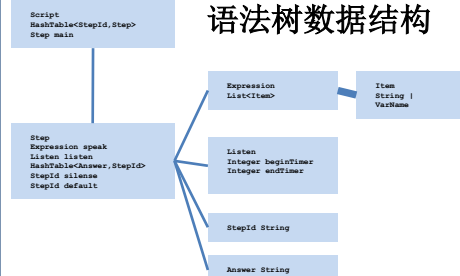
脚本文本
`test1.rsl`

不妨给这个脚本语言起个名字**RSL**:
Robot
Specification
Language

被测试的
robot
程序

Parser
语法分析

语法树数据结构

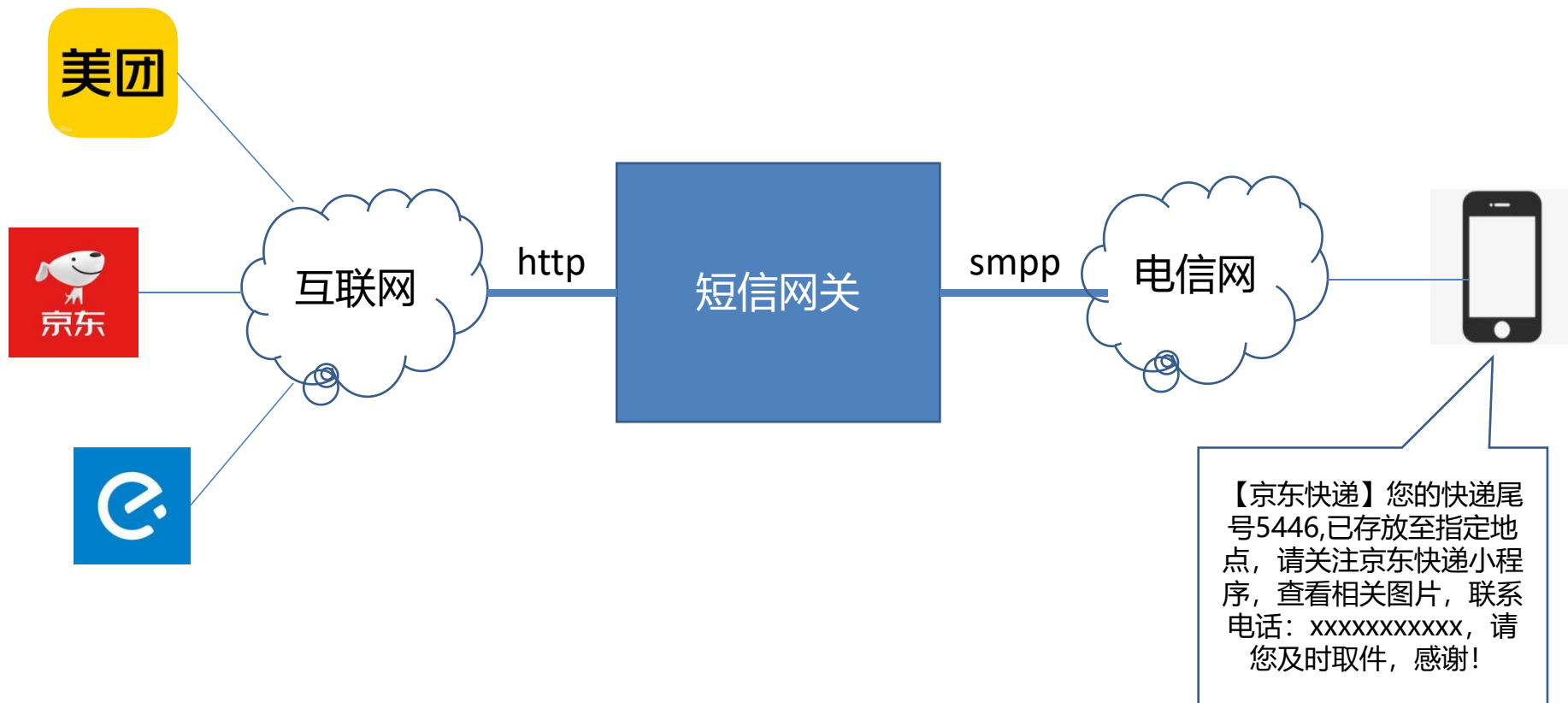


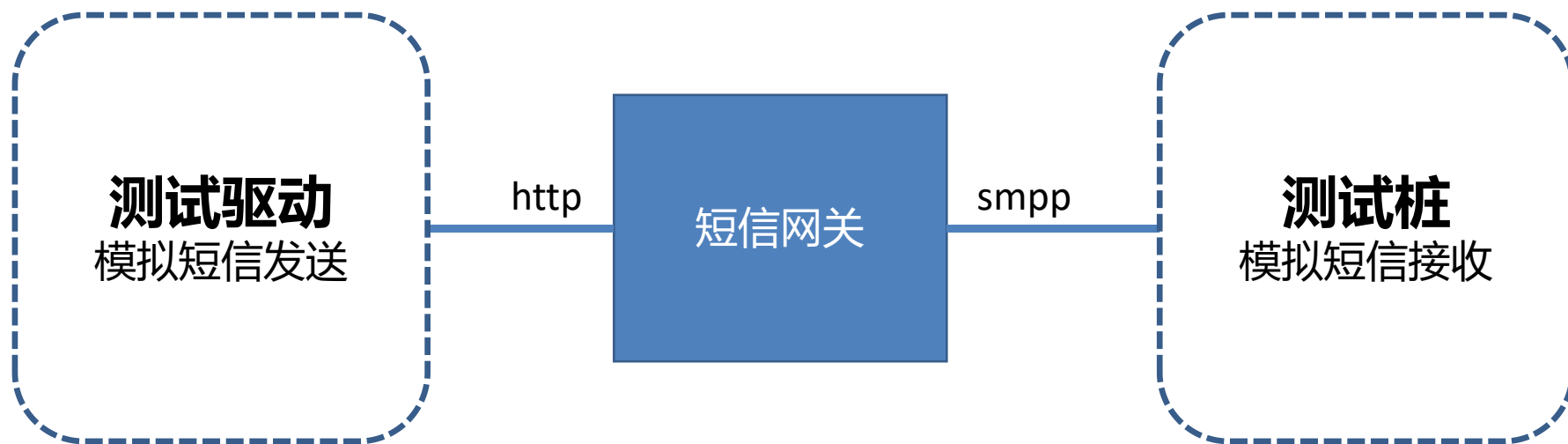
Interpreter
解释执行

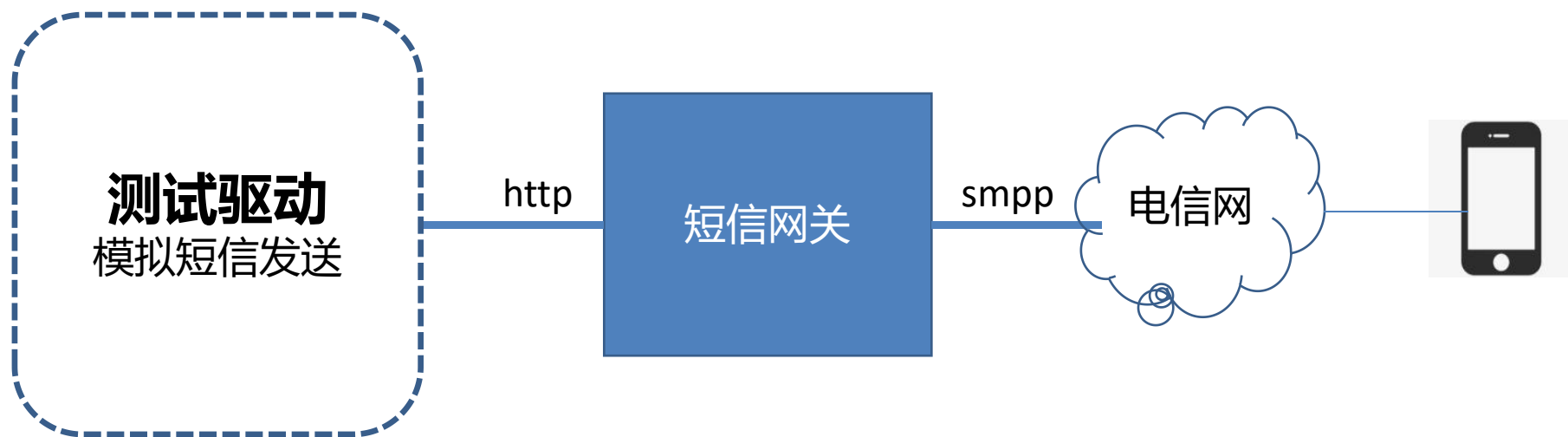
自然语言分析测试桩:
简单关键字匹配返回用户意愿

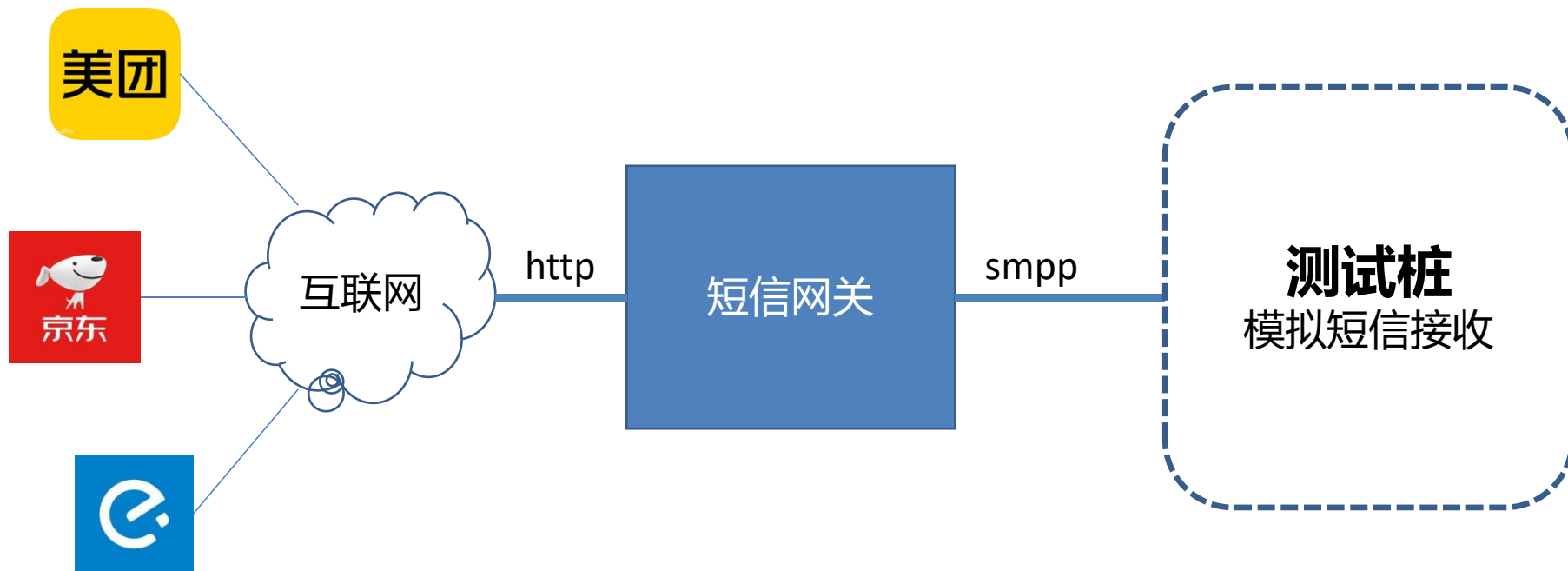
测试数据:
模拟应答文本集合

媒体服务器测试桩:
播放内容输出到文件
从测试数据文件读入应答文本









测试驱动：模拟短信发送

```
curl "http://127.0.0.1:27080/SendSMS" \  
-X POST \  
-d "{\  
    \"AppKey\": \"JDXXXX\", \  
    \"Token\": \"b1ffb6b5d22cd9f210fbc8b7fdaf0e19\", \  
    \"DestNumber\": \"138xxxxxxxx\", \  
    \"Type\": \"mormal\", \  
    \"Receipt\": \"off\", \  
    \"Conent\": \"【京东快递】您的快递尾号5446...\"\  
}"
```

测试驱动： 写成一个shell脚本

test_normal.sh

```
curl "http://127.0.0.1:27080/SendSMS" \  
-X POST \  
-d "{\  
    \"AppKey\": \"JDXXX\",\  
    \"Token\": \"b1ffb6b5d22cd9f210fbc8b7fdaf0e19\",\  
    \"DestNumber\": \"138xxxxxxx\",\  
    \"Type\": \"normal\",\  
    \"Receipt\": \"off\",\  
    \"Conent\": \"【京东快递】您的快递尾号5446...\"\  
}" > test_normal.result  
diff test_normal.result test_normal.ok
```

测试驱动：脚本携带参数

test_normal.sh xxxxxxxxxxxx

```
curl "http://127.0.0.1:27080/SendSMS" \  
-X POST \  
-d "{\  
    \"AppKey\": \"JDXXX\",\  
    \"Token\": \"b1ffb6b5d22cd9f210fbc8b7fdaf0e19\",\  
    \"DestNumber\": \"$1\",\  
    \"Type\": \"normal\",\  
    \"Receipt\": \"off\",\  
    \"Conent\": \"【京东快递】您的快递尾号5446...\"\  
}" > test_normal.result  
diff test_normal.result test_normal.ok
```

测试驱动：设计各种测试用例

- Json格式错误
- Json格式正确，但是缺少参数
- 不存在的AppKey，非法的Token
- 不同运营商的电话号码
- 格式错误的电话号码
- 需要回执，不需要回执
- 普通短信，闪信
- 短信不同的长度，需要拆分，不需要拆分
- ...

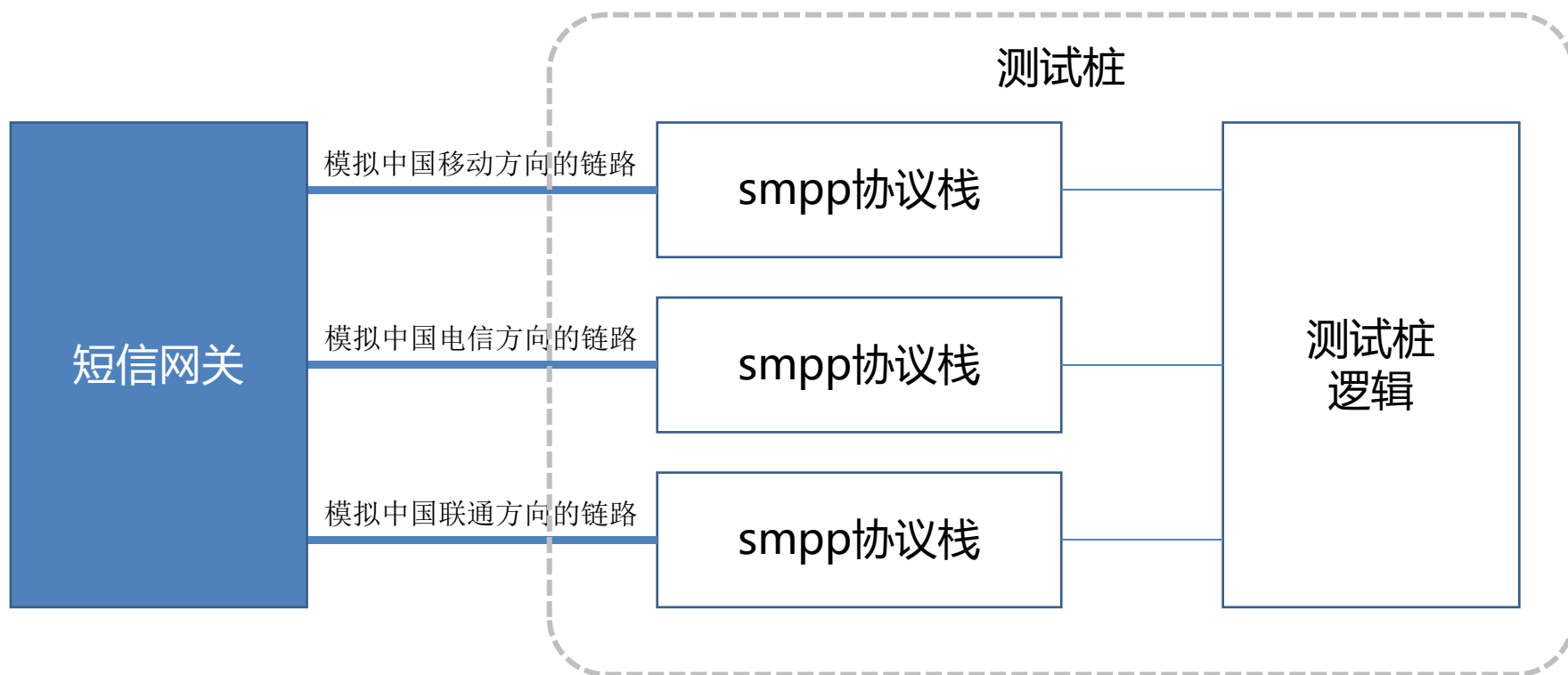
test_invalid_json.sh
test_missing_destnumber.sh
test_invalid_appkey.sh
test_china_telecom.sh
test_china_mobile.sh
test_no_receipt.sh
test_sms_push.sh
test_long_message.sh
...

测试驱动： 自动测试脚本

test_all.sh

```
test_invalid_json.sh xxxxxxxxxxxxxxxx
test_missing_destnumber.sh xxxxxxxxxxxxxxxx
test_invalid_appkey.sh xxxxxxxxxxxxxxxx
test_china_telecom.sh xxxxxxxxxxxxxxxx
test_china_mobile.sh xxxxxxxxxxxxxxxx
test_no_receipt.sh xxxxxxxxxxxxxxxx
test_sms_push.sh xxxxxxxxxxxxxxxx
test_long_message.sh xxxxxxxxxxxxxxxx
...
```

测试桩： 尽可能模拟实际的网络环境



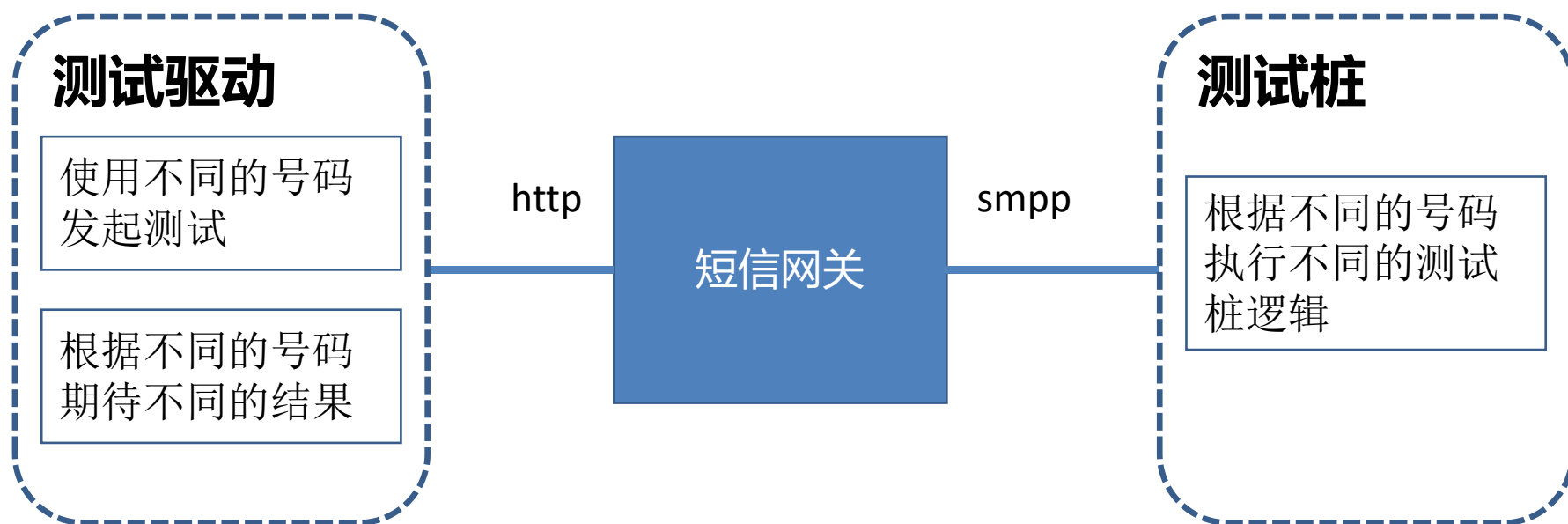
测试桩：各种测试桩逻辑

- 正常接收，返回正确应答，正常返回回执
- 不返回结果，测试超时的处理
- 返回错误应答，测试错误应答的处理
- 较长时间返回回执
- 不返回回执
- 返回的回执含有错误信息
- smpp链路失效
- ...

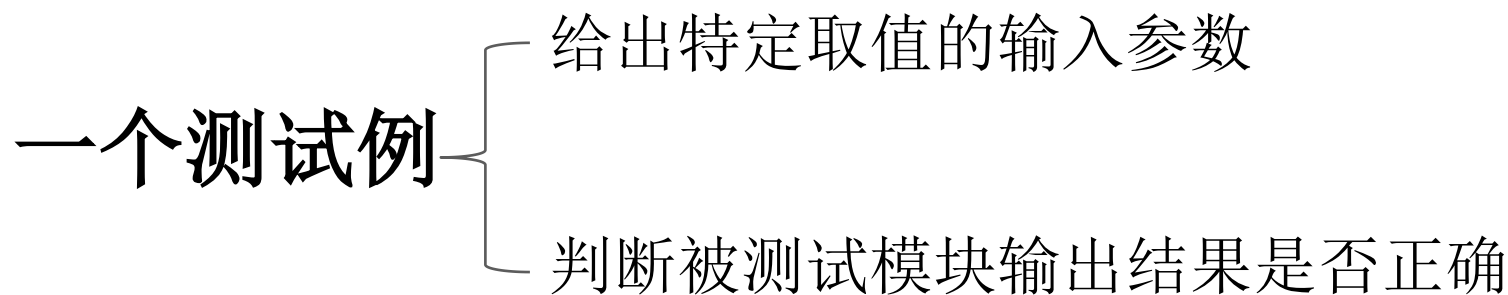
如何区分？

针对不同的号码
执行不同的逻辑

测试驱动和测试桩的配合



测试集的构建



测试集：

尽可能覆盖被测试模块的所有分支的一系列测试的集合。

测试集的构建 - 参数取值

对于测试驱动的输入参数，或者是测试桩的返回结果，至少需要包括这些取值：

- 正常值（例如70，80，95这些是“分数”这个参数的正常值）
- 边界值（例如0，100是“分数”这个参数的边界值）
- 非法取值（例如-1，500是“分数”这个参数的非法取值）

// 例如前面的测试str_reverse的测试集

```
test_str_reverse("abcde", "edcba"); // 正常值，奇数位
```

```
test_str_reverse("abcd", "dcba"); // 正常值，偶数位
```

```
test_str_reverse("a", "a"); // 边界值，1位
```

```
test_str_reverse("", ""); // 边界值，0位
```

```
// 想一下，还有哪些更加“邪恶”的取值？
```

例如前面的返回节假日的测试桩，应该返回哪些东西？

- 返回正常的节假日
- 不返回（测试http请求超时的情况）
- 返回的http code不是200，例如返回404等
- 返回的json字符串不符合规范，解析有问题
- 返回的json字符串解析之后缺少参数
- ...

测试集的构建 - 简单的结果判断

对于很多程序来说，输出正确与否可以简单用人工判断，这种测试的结果可以与人工构造的结果进行对比。

```
// 例如一个排序程序的测试集（人工很容易得出排序结果）  
test_sort({1,3,2}, {1,2,3});  
test_sort({5,100,2,0,-20}, {-20,0,2,5,100});  
...
```

```
// 例如前面的测试str_reverse的测试集（人工很容易得出字符串反转结果）  
test_str_reverse("abcde", "edcba");
```

即使是较为复杂的计算，对于边界值的运算也会相对简单，容易人工计算。例如坐标变换计算：

$$x = x' \cos \theta - y' \sin \theta$$

$$y = y' \cos \theta + x' \sin \theta$$

当 θ 为0, $\pi/2$, π 等值的时候，很容易手工计算。

测试集的构建 - 复杂的结果判断

有些程序的输出正确与否，很难人工判断，如何判断它的输出是否正确？可以尝试如下方法：

- 如果存在同类程序，可以用其它已经被认为正确的程序计算正确的结果作为这个程序的计算结果的判断依据。
- 对于存在逆运算的程序，可以判断参数通过运算和逆运算之后是否能还原。例如：
 - 加密解密
 - 压缩解压缩
 - 对数运算和指数运算
- 将计算结果绘制成图比较直观地展示出来，可以人工判断结果的合理性
 - 例如，运动轨迹的平滑程序，平滑之前之后的轨迹数据很难人工判断，将每个点在坐标系中绘制出来就比较容易观察了
- 测试新版本的时候，使用老版本已经通过测试的测试集。
 - 保留老版本的测试集，以及测试集的运行结果，新版本用同样的测试集产生的结果与之比较，即回归测试。
- 使用两种不同的算法计算，比较它们的结果是否一致。

测试自动化(Test Automation)

用手工的方式测试，乏味，工作量大，不可靠。

应该把时间用来精心构造测试自动化程序或脚本，构造好了之后，可以无数次运行它。

测试自动化是真正实现回归测试的最好的手段。任何一个细微的修改都可以进行一次回归测试，因为测试自动化脚本可能只需要几分钟甚至几秒钟就可以运行完毕，报告回归测试是否成功。

随着程序功能的增加，应该不断地完善这个测试自动化程序或脚本。

如果发现了这个自动化程序或脚本没有覆盖的错误，应该及时将产生这个错误的用例添加到测试自动化程序或者脚本中。

str.c: 实现了str_reverse函数的c文件

main.c: 真正的main函数所在的文件

test.c: 实现了test_str_reverse函数以及测试用main函数所在的文件

makefile

```
exe: str.o main.o
    cc -o exe str.o main.o
str.o: str.c
    cc -c str.c
main.o: main.c
    cc -c main.c
```

包含自动测试的makefile

```
exe: str.o main.o test.o
    cc -o exe_test str.o test.o
    ./exe_test
    rm -f exe_test
    cc -o exe str.o main.o
str.o: str.c
    cc -c str.c
main.o: main.c
    cc -c main.c
test.o: test.c
    cc -c test.c
```

假设有一个"ka"程序，它的功能是对一个文件做处理，输出处理结果：

```
ka ka_data.1 > out1
```

假设旧版本的"ka"程序叫做old_ka，有一系列的测试文件，都已经测试的没有问题，例如这些测试文件叫做ka_data.1， ka_data.2等等若干个。

现在对"ka"做了修改，新版本叫做new_ka，那么如何做回归测试呢？

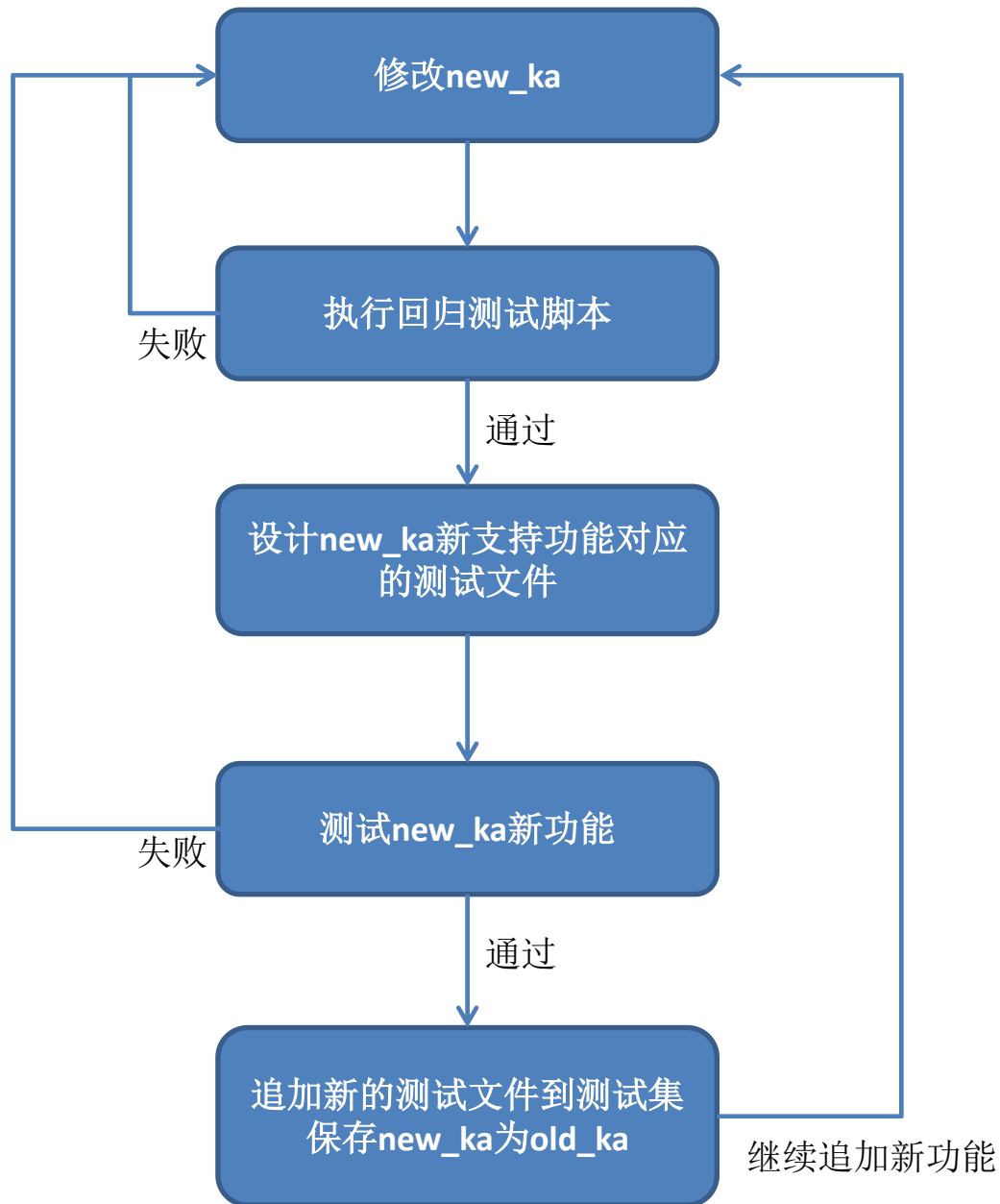
可以写这样一个shell程序：

```
for i in ka_data.*; do          # 对所有的测试文件循环做
    old_ka $i > out1             # old_ka处理文件i，输出结果到out1
    new_ka $i > out2             # new_ka处理相同的文件i，输出结果到out2
    if ! cmp -s out1 out2; then  # 比较out1和out2
        echo $i: BAD            # 如果不相同，则输出文件名和"BAD"
    fi
done
```

如果回归测试通过，那么上面的脚本不会有任何输出。

如果某个测试文件new_ka处理的结果和old_ka不同，那么说明回归测试失败，报出问题所在。

回归测试通过之后，如果new_ka增加了新功能，则编辑新的测试文件，例如ka_data.300（假设之前已经有了299个测试文件）



awk用法简介

```
echo 3 5 | awk '{print $1}' # 显示3
echo 3 5 | awk '{print $2}' # 显示5
echo 3 5 | awk '{i = 1; print $i}' # 显示$1, 即3
echo 3 5 | awk '{i = 1; print i}' # 显示i, 即1
echo 3 5 | awk '{i = 1; print $i++}' # 先显示$1, 即3, 并将$1加了1
```

awk的其中一个测试脚本

```
# field increment test: $i++ means ($i)++, not $(i++)
echo 3 5 | awk '{i = 1; print $i++; print $1, i}' >out1
echo '3
4 1' >out2 # correct answer
if ! cmp -s out1 out2 # output are different
then
    echo 'BAD: field increment test failed'
fi
```

“我们为awk设计了大约一千个测试，整个测试集用一个命令就可以运行，如果所有的东西都没有问题，那么测试将不产生任何输出。”

思考题：robot程序如何构建自动测试脚本？

提示：

构造各种不同的测试RSL脚本...

构造各种不同的应答文本数据...

构造不同的命令行来驱动...

如何关联他们？

性能测试(Performance Testing)

性能测试用于验证程序运行正确的前提下，不同环境下的性能指标。

性能指标

并发数量：（标准指标术语）

程序可以同时为多少用户提供服务？

例如通常所说的，一个电信设备支持多少“线”

单位时间服务请求数量：

每秒钟可以允许多少个服务请求被处理？

例如电信领域中常用的CAPS(Call Attempt Per Second)单位

响应时间：

在上面两个指标高位运行情况下，消息的响应时间。

思考题： 对一个处理呼叫的电信设备来说，并发数量 / CAPS = ?

性能测试 - 目的

对程序性能有较高要求的一些案例：

- 12306售票
- 电商促销活动
- 各类电信业务

验证程序是否能够达到用户要求的性能指标

- 用户需要完整的测试报告
- 厂商需要对自己程序的性能指标有所了解

为硬件配置提供数据参考

- 依据程序已有的性能数据，依据硬件设备公开的性能指标比较，可以大致推算出更高的性能要求所需的更高的硬件配置。

为性能优化提供依据

- 性能优化，是一个不断地性能测试->寻找瓶颈->优化修改->性能测试的过程。

性能测试 - 搭建环境

被测试程序的安装:

- 尽可能选择与目标设备相同的硬件设备。
- 安装与目标设备相同版本的操作系统。
- 被测程序的版本与目标设备的要求相同。
- 关注网卡的带宽，通过计算保证不要让网卡带宽成为瓶颈。

测试驱动和测试桩的安装:

- 测试桩和测试驱动需要和被测试程序分离，不要位于相同的设备。
- 测试桩和测试驱动本身不能成为性能瓶颈。
- 选择被测程序的典型流程，按照这个流程设计测试桩和测试驱动的参数。

一个例子:



性能测试 - 测试驱动的功能要求

测试驱动程序需要具备这些功能：

- 可以设定请求参数，可以设置每次请求的持续时间
- 可以设定请求频率（每秒多少次请求）
- 可以设定测试总时长。
- 可以设定测试请求总次数。

一个例子：

```
// 设置测试脚本以及参数，test.usl脚本将会发起一个呼叫，持续60秒的时长  
masstest -a 'test.usl makecall 60'
```

```
masstest -c 100 // 设置测试频率，每秒钟调用100次上述脚本
```

```
masstest -n 1000000 // 设置测试脚本调用总次数
```

```
masstest start // 开始测试
```

上述测试发起之后：被测试程序将会每秒接收多少次呼叫请求（CAPs）？同时会有多少个并发呼叫？测试会多长时间结束？

性能测试 - 测试过程

性能指标由低到高进行测试：

- 例如目标性能指标为100CAPS，可以先测试10CAPS。
- 观察系统运行100%正常，再逐渐增加，测试20CAPS， 50CAPS， 100CAPS

测试期间观察的内容：

- 查看功能是否正常：日志中是否有异常输出，统计数据是否合理。
- 查看CPU的占用情况：总的CPU占用率，各个进程，线程的CPU占用率，CPU占用是否均衡。
- 查看内存占用：系统总体内存占用，每个进程的虚存占用，它们是否在不断增长（如果是，则可能有内存泄露）

测试记录：

- 记录当前的测试环境
- 记录当前的测试条件
- 与测试条件对应的系统资源占用情况

性能测试 - 测试过程中CPU的查看

```
Cpu(s):  0.3%us,  0.3%sy,  0.0%ni, 99.3%id,  0.0%wa,  0.0%hi,  0.0%si,  0.0%st
```

所有CPU的总体情况

us: (user)用户态CPU占用百分比
sy: (system)内核态CPU占用百分比
ni: (nice)如果使用nice降低过优先级，低优先级的进程占用CPU百分比
id: (idle)空闲百分比
wa: (wait)等待I/O的CPU占用百分比
hi: (hardware-interrupt)处理硬件中断占用CPU的百分比
si: (software-interrupt)处理软件中断占用CPU的百分比
st: (steal-time)虚拟机窃取的时间占用的百分比

分核统计:

```
Cpu0  :  0.7%us,  0.0%sy,  0.0%ni, 99.3%id,  0.0%wa,  0.0%hi,  0.0%si,  0.0%st
Cpu1  :  0.3%us,  0.3%sy,  0.0%ni, 99.3%id,  0.0%wa,  0.0%hi,  0.0%si,  0.0%st
Cpu2  :  0.3%us,  0.0%sy,  0.0%ni, 99.7%id,  0.0%wa,  0.0%hi,  0.0%si,  0.0%st
Cpu3  :  0.3%us,  0.3%sy,  0.0%ni, 99.3%id,  0.0%wa,  0.0%hi,  0.0%si,  0.0%st
Cpu4  :  0.3%us,  0.3%sy,  0.0%ni, 99.3%id,  0.0%wa,  0.0%hi,  0.0%si,  0.0%st
Cpu5  :  0.3%us,  0.7%sy,  0.0%ni, 99.0%id,  0.0%wa,  0.0%hi,  0.0%si,  0.0%st
Cpu6  :  0.3%us,  0.3%sy,  0.0%ni, 99.3%id,  0.0%wa,  0.0%hi,  0.0%si,  0.0%st
Cpu7  :  0.3%us,  0.0%sy,  0.0%ni, 99.7%id,  0.0%wa,  0.0%hi,  0.0%si,  0.0%st
```

分进程统计（进程占用的单CPU百分比）：

PID	USER	PR	NI	VIRT	RES	SHR	S	%CPU	%MEM	TIME+	COMMAND
172876	zhib1	20	0	68296	7900	3232	S	0.7	0.0	82:59.70	uniframe
39	root	20	0	0	0	0	S	0.3	0.0	0:44.33	events/4
3258	oracle	20	0	1234m	16m	14m	S	0.3	0.1	0:10.02	oracle

性能测试 - 测试过程中内存的查看

```
Mem: 16084392k total, 14285896k used, 1798496k free, 426200k buffers
Swap: 8110076k total, 0k used, 8110076k free, 12379620k cached
```

内存的总体情况

Mem_total: 内存总量, 16G
 Mem_used: 正在使用: 14.3G
 Mem_free: 空闲, 未分配: 1.8G
 Mem_buffers: 用作缓存, 但是可以随时释放的可用内存, 0.4G
 Swap_total: Swap总量, 8G
 Swap_used: Swap正在使用, 0
 Swap_free: Swap未分配: 8G
 Swap_cached: 用作Swap缓存, 可以随时释放的可用内存, 12.4G

全部可用内存 = Mem_free + Mem_buffers + Swap-cached

PID	USER	PR	NI	VIRT	RES	SHR	S	%CPU	%MEM	TIME+	COMMAND
172876	zhib1	20	0	68296	7900	3232	S	0.7	0.0	82:59.70	uniframe
39	root	20	0	0	0	0	S	0.3	0.0	0:44.33	events/4
3258	oracle	20	0	1234m	16m	14m	S	0.3	0.1	0:10.02	oracle

分进程统计

VIRT: 进程代码, 数据, 堆栈所占用的全部虚存大小 (查看内存泄露主要关注此项)
 RES: 进程所使用的物理内存的大小
 SHR: 进程所使用的共享内存的大小
 %MEM: 进程所占用的物理内存占内存总量的百分比, 即: RES / Mem_total

性能测试 - 测试报告

测试环境总体说明：

- 硬件环境描述：
 - 设备型号
 - CPU主频，数量
 - 内存容量
 - 网卡带宽
- 操作系统版本
- 被测测试程序版本
- 测试典型流程

针对每一次测试说明：

- 测试参数：
 - 请求频率，并发数量，测试时长（测试数量）
- 测试结果：
 - 程序运行是否正常（无报错，无告警，响应时间正常）
 - 关键统计数据（关键步骤统计，收发消息数量统计）
- 测试过程中资源占用情况：
 - CPU，内存，磁盘，网络带宽的占用情况
 - 瓶颈进程的CPU内存占用

压力测试(Stress Testing)

压力测试（或者称为破坏性测试，强度测试，应力测试），用于验证程序在极端情况下的稳定性和可靠性表现，从而发现程序在极端情况下的问题，做好保护措施，避免在实际运行中出现事故级别的问题。



提供大用户量服务的程序，尤其要避免雪崩效应的出现。

压力测试是查出程序中这类弱点的重要手段。

压力测试 - 测试极端情况

制造输入异常的情况，例如：

- 构造大量异常输入内容，例如特别长的字符串，特别大的数字。
- 构造高频率的输入请求，超过系统的性能指标。

制造资源紧缺的情况，例如：

- 同时启动另一个占用大量CPU和大量内存的程序。
- 同时启动另一个占用网络带宽的程序。
- 同时制造磁盘占用接近100%的情况。

制造硬件故障情况，例如：

- 拔掉网线再插上。
- 断电其中某台设备再重新上电。

长时间运行的考验，例如：

- 输入请求维持高位情况下运行24小时甚至更长时间

压力测试 - 可靠性表现

对于异常输入参数:

- 异常输入参数, 应该忽略或者拒绝, 应该记录日志。
- 不应该影响正常服务。

对于超过性能指标的请求:

- 应该有流量控制机制按比例过滤超过性能指标的请求。
- 不应该暂停服务。

对于硬件故障:

- 当故障消除之后, 应该立即提供服务。
- 故障消除之后一般会伴随大量的访问请求, 应该有保护措施。

长时间运行:

- 不应该有累积的故障隐患。
- 服务质量不应该随着时间变化下降。